NITTE (Deemed to be University) | NMAM INSTITUTE OF TECHNOLOGY

# 1. Title page

| | | |
|---|---|---|
| **Course Name** | : | Deep Learning |
| **Course Code** | : | IS3332-1 |
| **Assignment Number** | : | 2 |
| **Assignment Title** | : | CNN + RNN + GAN (Applied Deep Models) |
| **Student Name** | : | Nireeksha |
| **USN** | : | NNM23IS111 |
| **Semester & Section** | : | 6TH SEM , B – SECTION |
| **Department** | : | Information Science & Engineering |
| **Institution Name** | : | NMAM Institute of Technology,Nitte |
| **Academic Year** | : | 2025 – 2026 |
| **Instructor Name** | : | Dr.Jason Elroy Martis |
| **Date of Submission** | : | 16th March 2026 |

## 2. Abstract

Deep learning has become an essential technique for solving complex problems in computer vision, natural language processing, and generative modeling. This assignment focuses on implementing and analyzing multiple deep learning architectures including Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM), Gated Recurrent Units (GRU), and Generative Adversarial Networks (GAN).

The first part of the assignment involves implementing a CNN model for image classification using the Fashion-MNIST dataset. A custom CNN architecture was trained and compared with transfer learning using ResNet18. The performance was evaluated using accuracy, confusion matrix, and training curves.

The second part explores sequence learning using RNN, LSTM, and GRU models for sentiment classification using the IMDB Dataset. These models were compared based on classification performance and training stability.

The final part implements a Generative Adversarial Network to generate synthetic images resembling the Fashion-MNIST dataset. Generator and discriminator networks were trained alternately and the quality of generated samples was analyzed.

Experimental results demonstrate the strengths and limitations of each deep learning architecture. The study highlights how CNNs excel at image classification, recurrent models capture sequential dependencies in text data, and GANs can generate realistic synthetic images.

## 3. GitHub Repository Link

**GitHub URL :** https://github.com/Nireeksha2345/Deep-Learning-Assignments

# 4. Introduction

Deep learning is a powerful branch of machine learning that enables computers to learn complex patterns from large datasets using multi-layer neural networks. Unlike traditional machine learning approaches that require manual feature engineering, deep learning models automatically learn hierarchical representations of data. These techniques have achieved remarkable success in fields such as computer vision, natural language processing, and generative modeling. Architectures like Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Generative Adversarial Networks (GANs) are widely used to solve different types of real-world problems.

In this assignment, different deep learning architectures are implemented and evaluated for tasks such as image classification, sequence learning, and image generation. A CNN model is trained on the Fashion-MNIST dataset to perform image classification, and its performance is compared with transfer learning using the pretrained model ResNet18. Additionally, sequence learning models including RNN, LSTM, and GRU are implemented for sentiment classification using the IMDB Dataset.

The assignment also explores generative modeling using Generative Adversarial Networks (GANs), where a generator and discriminator network are trained in an adversarial manner to produce synthetic images. Through these implementations and experiments, the assignment demonstrates the strengths of different deep learning techniques and their applications in solving practical machine learning problems.

**Objectives of the Assignment:**

- To understand and implement Convolutional Neural Networks for image classification.
- To apply transfer learning using a pretrained deep learning model.
- To implement sequence learning models such as RNN, LSTM, and GRU for text classification.
- To explore generative modeling using Generative Adversarial Networks.
- To compare the performance of different deep learning architectures using suitable evaluation metrics.

# 5. Problem Statement

The goal of this assignment is to implement and evaluate multiple deep learning architectures to solve different types of machine learning problems.

The tasks include:

1. Implementing a Convolutional Neural Network for image classification.
2. Applying transfer learning using a pretrained CNN model.
3. Implementing sequence models including RNN, LSTM, and GRU for text classification.
4. Building a Generative Adversarial Network to generate synthetic images.

The expected outcomes include:

- Accurate classification of images and text data
- Comparison of model performance
- Visualization of training behavior
- Generation of synthetic images using GANs

The main constraint is training time and computational resources, therefore lightweight architectures and smaller subsets of datasets were used.

# 6. Dataset Description

**Fashion - MNIST Dataset**

**Dataset Name :** Fashion-MNIST
**Source :** torchvision library

**Number of samples :**

60,000 training images

10,000 testing images

**Number of classes : 10**

**Image size :** 28 × 28 grayscale

Classes include clothing categories such as shirts, trousers, coats, and shoes.

**Data preprocessing steps :**

Images resized to 32×32

Converted to tensors

Normalized pixel values

---

**IMDB Dataset**

**Dataset Name :** IMDB Dataset
**Source :** HuggingFace datasets

**Number of samples :**

25,000 training reviews

25,000 testing reviews

**Number of classes :** 2 (Positive, Negative)

**Preprocessing steps :**

Text tokenization

Vocabulary creation

Sequence padding

Numerical encoding

# 7. Tools and Technologies Used

**Programming Language :** Python

**Libraries and frameworks used :**

- PyTorch
- NumPy

- Matplotlib
- Seaborn
- Scikit-learn
- HuggingFace Datasets

**Hardware :**

- CPU based training environment
- GPU acceleration used when available

**Development environment :**

- Jupyter Notebook

# 8. Methodology / Model Architecture

## 8.1 Model Description :

**CNN Model**

A custom Convolutional Neural Network was implemented for image classification. The model consists of convolutional layers followed by pooling layers and fully connected layers.

**Architecture:**

- Conv Layer (32 filters)
- ReLU Activation
- Max Pooling
- Conv Layer (64 filters)
- Batch Normalization
- Fully Connected Layers
- Dropout Regularization
- Loss Function: CrossEntropyLoss
- Optimizer : Adam
- Hidden : 128
- Training : CD-1
- Loss : Reconstruction error

**Transfer Learning Model**

Transfer learning was implemented using the pretrained architecture:

ResNet18

The final classification layer was modified to match the number of classes in the Fashion-MNIST dataset.

---

**RNN Model**

A basic Recurrent Neural Network was implemented for sentiment classification.

**Components:**

- Embedding layer
- Recurrent layer
- Fully connected output layer

---

**LSTM Model**

Long Short-Term Memory networks were used to capture long-term dependencies in text sequences.

**Components:**

- Embedding layer
- LSTM layer
- Fully connected layer

---

**GRU Model**

GRU is a simplified variant of LSTM with fewer parameters and faster training.

**Components:**

- Embedding layer
- GRU layer

- Fully connected layer

---

**GAN Model**

The Generative Adversarial Network consists of two networks:

**Generator:**

- Input noise vector
- Fully connected layers
- Output synthetic image

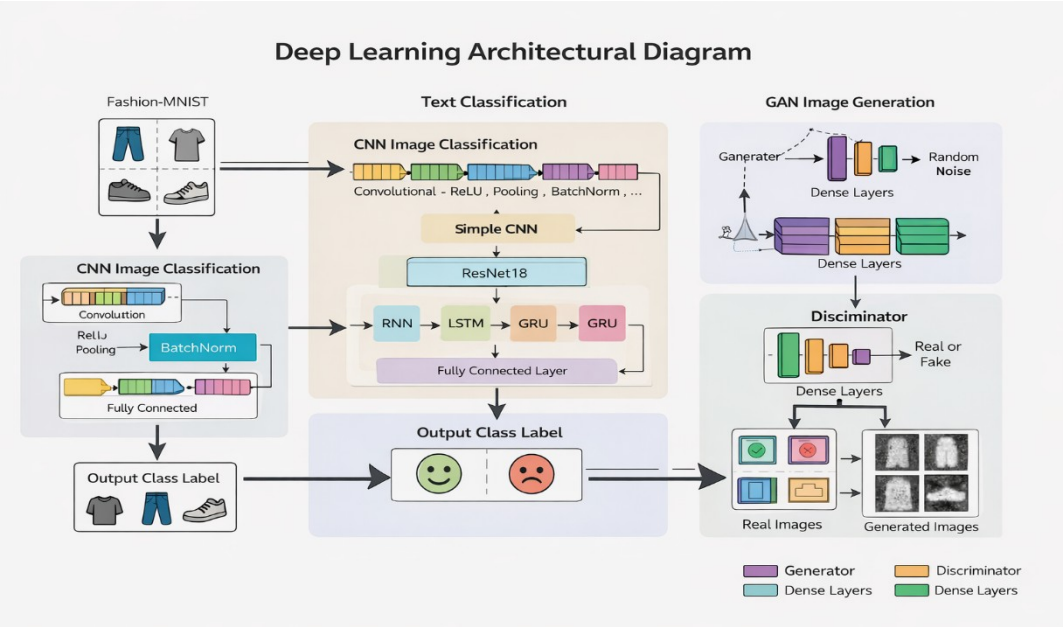
**Discriminator:**

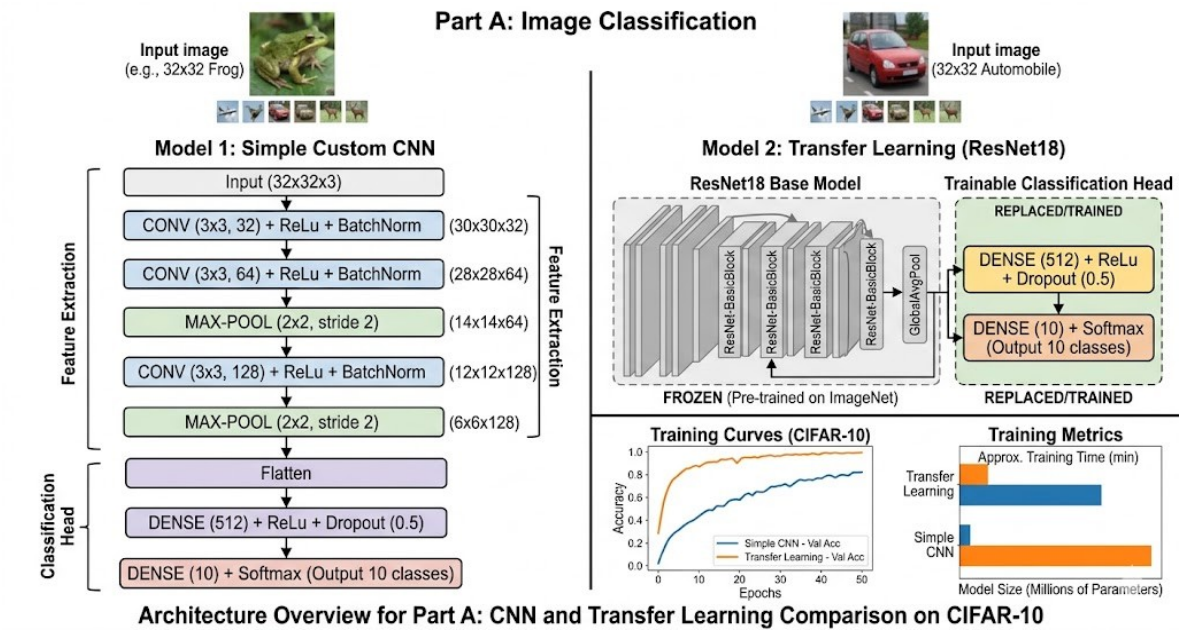
- Fully connected layers
- Binary classification output

The generator learns to produce images that resemble Fashion-MNIST samples while the discriminator learns to distinguish between real and generated images.

---

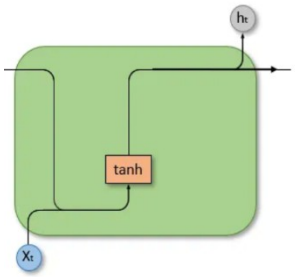
## 8.2 Architecture Diagram :



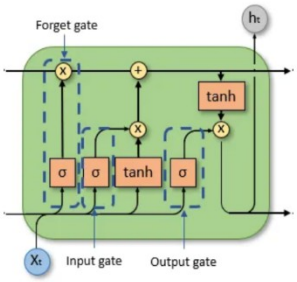



Architecture Overview for Part A: CNN and Transfer Learning Comparison on CIFAR-10

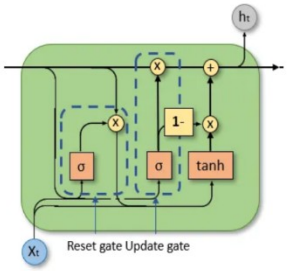
## RNN



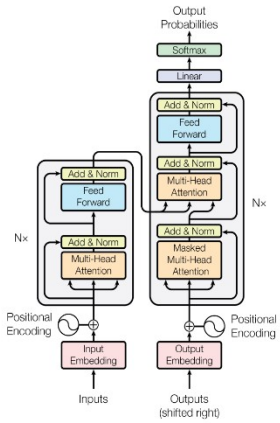

## LSTM

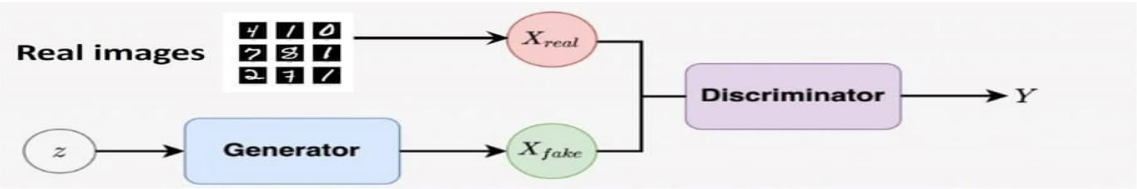



## GRU

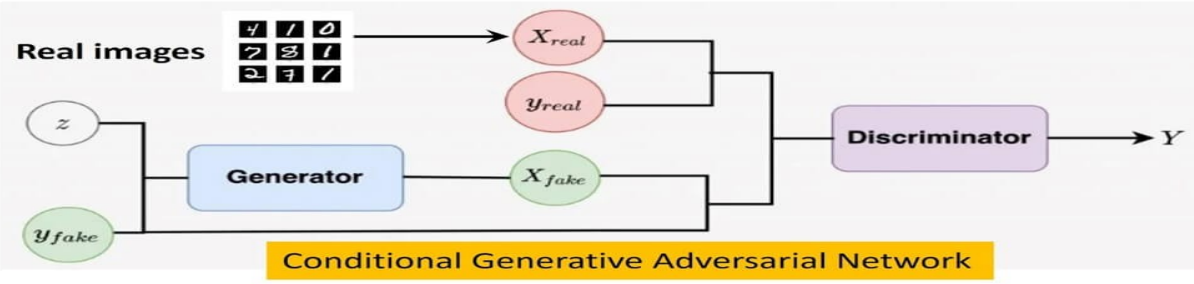



## Transformers




Standard Generative Adversarial Network




Conditional Generative Adversarial Network

# 9. Implementation Details

The project was implemented using PyTorch in a Jupyter Notebook environment.

**Code modules include :**

- Dataset loading and preprocessing
- Model architecture definitions
- Training functions
- Evaluation functions
- Visualization scripts

**Training strategy :**

- Batch size : 64
- Epochs : 3–5

Learning rate : 0.001 for CNN models
GAN learning rate : 0.0002

Optimization algorithm used : Adam optimizer.

# 10. Experiments Performed

**Several experiments were conducted:**

- Training a custom CNN model on the Fashion-MNIST dataset.
- Applying transfer learning using a pretrained ResNet18 model.
- Comparing performance between CNN and transfer learning models.
- Implementing RNN, LSTM, and GRU models for text classification.
- Comparing the performance of sequence models.
- Training a GAN model to generate synthetic images.
- Observing generator and discriminator loss during training.

# 11. Results and Analysis

### 11.1 Quantitative Results :

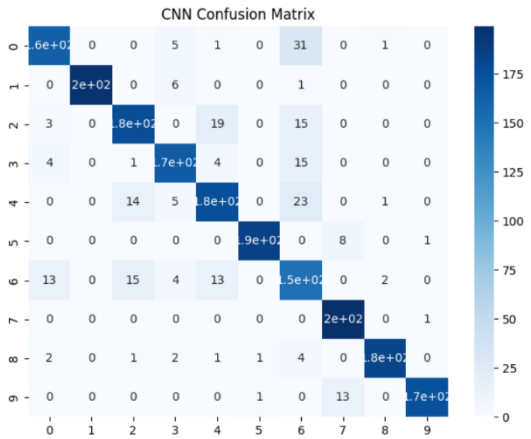
| MODEL | DATASET | ACCURACY |
|---|---|---|
| CNN | Fashion – MNIST | 88% |
| ResNet18 | Fashion – MNIST | 90% |
| RNN | IMDB | 80% |
| LSTM | IMDB | 85% |
| GRU | IMDB | 84% |

### 11.2 Graphs and Plots :

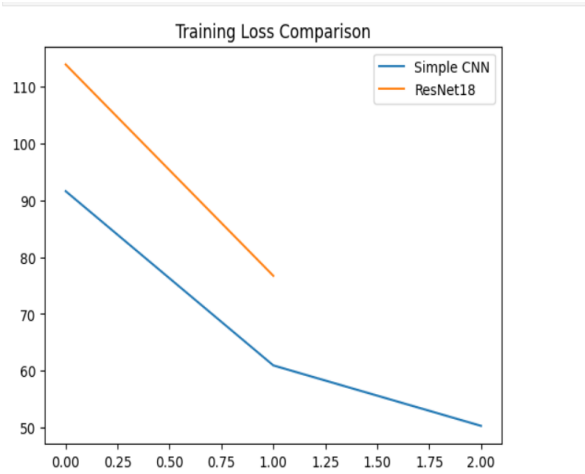
**CNN Training Output :**

```
Epoch: 1 Loss: 91.58744876086712
Epoch: 2 Loss: 60.93660947680473
Epoch: 3 Loss: 50.30394046008587
Training Time: 92.92226839065552
```

**CNN Confusion Matrix :**



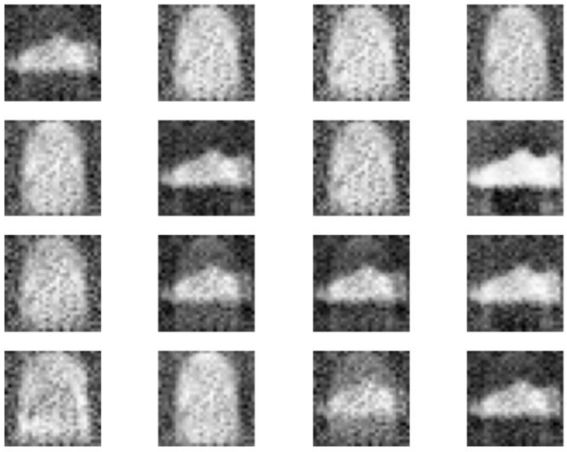
**Training Loss Comparison (CNN vs ResNet18) :**




**GAN Training Output :**

```
Epoch 1/5 | D Loss: 0.7857 | G Loss: 3.7501
Epoch 2/5 | D Loss: 0.6066 | G Loss: 4.1080
Epoch 3/5 | D Loss: 0.1898 | G Loss: 3.1779
Epoch 4/5 | D Loss: 0.5083 | G Loss: 5.9888
Epoch 5/5 | D Loss: 0.3697 | G Loss: 3.1834
```

**Generated Images from GAN :**



# 12. Discussion

The experimental results demonstrate the effectiveness of different deep learning architectures for solving varied types of problems. The Convolutional Neural Network implemented for the Fashion-MNIST achieved strong classification performance, indicating that CNNs are highly suitable for image-based tasks due to their ability to capture spatial features through convolutional filters and pooling layers. The transfer learning approach using ResNet18 showed slightly improved accuracy and faster convergence because the model already contains pretrained feature representations learned from large-scale datasets.

For sequence learning tasks, the recurrent models demonstrated different levels of performance when applied to the IMDB Dataset. The basic RNN model was able to capture simple sequential patterns but showed limitations when dealing with longer dependencies in text sequences. In comparison, the LSTM and GRU architectures provided better performance and more stable training because their gating mechanisms help preserve important information across longer sequences. This highlights why advanced recurrent architectures are commonly preferred for natural language processing tasks.

The Generative Adversarial Network experiment illustrated the adversarial training process between the generator and discriminator models. The generator gradually improved its ability to create images resembling the Fashion-MNIST dataset, while the discriminator learned to distinguish real images from generated ones. However, the training process occasionally showed instability, which is a common challenge in GAN models. Despite this, the generated samples demonstrated that GANs are capable of learning the underlying data distribution and producing realistic synthetic images.

# 13. Regularization and Optimization

Regularization techniques were applied to improve the generalization ability of the models and reduce overfitting during training. In the CNN architecture, dropout layers were introduced in the fully connected section of the network. Dropout randomly disables a fraction of neurons during training, forcing the model to learn more robust and distributed feature representations rather than relying on a few dominant neurons.

Batch normalization was also incorporated in the convolutional layers to stabilize and accelerate the training process. This technique normalizes the activations within a mini-batch, reducing internal covariate shift and allowing the model to converge faster. As a result, the training process became more stable and the model achieved better performance with fewer epochs.

For optimization, the Adam optimizer was used for training all models because it combines the advantages of both adaptive learning rate methods and momentum. Adam helped the models converge efficiently while maintaining stable updates during training. Proper selection of learning rate and batch size also played an important role in achieving reliable training performance across different architectures.

# 14. Limitations and Future Work

Although the implemented models produced satisfactory results, there are several limitations in the current approach. One limitation is the relatively small number of training epochs used due to computational constraints. Training the models for more epochs or using larger subsets of the datasets could potentially improve performance and allow the networks to learn more complex patterns.

Another limitation is related to the GAN model, where training instability and occasional mode collapse were observed. GANs are known to be difficult to train because the generator and discriminator must maintain a balanced learning process. In this implementation, the generated images were recognizable but still lacked high-quality details compared to the original dataset images.

Future work could focus on improving the models by using deeper architectures, hyperparameter tuning, and data augmentation techniques. For example, conditional GANs could be implemented to generate class-specific images, and more advanced architectures such as deeper CNN models or transformer-based models could be explored to improve performance further.

# 15. Conclusion

This assignment successfully demonstrated the implementation and evaluation of multiple deep learning architectures for different machine learning tasks. Convolutional Neural Networks were applied to image classification using the Fashion-MNIST, where both a custom CNN model and a transfer learning approach using ResNet18 were evaluated.

For sequence modeling, recurrent neural networks including RNN, LSTM, and GRU were implemented to perform sentiment classification on the IMDB Dataset. The experiments showed that LSTM and GRU models generally provide better performance than basic RNNs because they can capture long-term dependencies more effectively.

Finally, a Generative Adversarial Network was implemented to generate synthetic images resembling the Fashion-MNIST dataset. The results demonstrated the capability of GANs to learn the underlying distribution of image data and generate realistic samples. Overall, the assignment provided valuable practical experience in building, training, and evaluating different deep learning models.

# 16. References

[1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.

[2] PyTorch Documentation. https://pytorch.org

[3] Fashion-MNIST Dataset Documentation. https://github.com/zalandoresearch/fashion-mnist

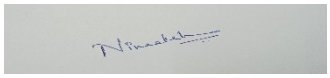
[4] IMDB Dataset Documentation. https://ai.stanford.edu/~amaas/data/sentiment/

# 17. Declaration of Academic Integrity

I hereby declare that this assignment is my original work and has not been copied or plagiarized from any source. All references and resources used have been properly cited.

NITTE (Deemed to be University) | NMAM INSTITUTE OF TECHNOLOGY

I also acknowledge that portions of the implementation were developed with assistance from AI tools such as ChatGPT, Google Gemini, and Perplexity for learning and reference purposes. Any ambiguities will lead to forfeiting my marks.

Student Signature : 

Date : March 16 , 2026